Non-Differentiable Learning of Quantum Circuit Born Machine with Genetic Algorithm

Alexei Kondratyev
Department of Mathematics, Imperial College London,
e-mail: a.kondratyev@imperial.ac.uk

Abstract
The quantum circuit Born machine (QCBM) is a generative quantum machine learning model that can be efficiently trained and run on the NISQ-era quantum processors. It is likely that the QCBM will be one of the first quantum machine learning models to find productive applications in quantitative finance as a powerful market generator. In this paper we test QCBM performance on a dataset of spot FX log-returns (heavy-tailed distribution) as well as a specially constructed mixture of normal distributions (which models spikey light-tailed distributions). The QCBM has greater expressive power than comparable classical neural networks such as the restricted Boltzmann machine (RBM) and, therefore, has potential to demonstrate quantum advantage by generating high-quality samples from the learned empirical distribution of the market risk factors while using less computational resources than its classical counterpart.

However, efficient training of the QCBM remains a challenging problem. The traditional differentiable learning approach may not work well when the loss function is highly non-smooth. In such cases it may be more efficient to use the non-differentiable learning methods. This paper proposes a non-differentiable learning approach to the training of the QCBM based on a genetic algorithm (GA). The paper also presents the results of the numerical experiments which compare the performance of the QCBM trained with a GA against the performance of the equivalent classical RBM and investigates the question of GA convergence as a function of QCBM architecture and the choice of algorithm's hyperparameters.

Keywords
generative models, quantum circuit Born machine, genetic algorithm, restricted Boltzmann machine, parametrized quantum circuit

JEL classification: C63, G17

1 Introduction

Generative models form one of the most important classes of unsupervised machine learning techniques, with numerous applications in such diverse fields as finance (generation of synthetic market data [1,2]), medicine (data anonymization [3,4]), computer vision (image-to-image translation [5]), engineering (design optimization [6]), and physics (Ising models with many-spin interactions [7,8]), to name just a few.

Although there are many well-studied types of generative models, two neural network-based non-parametric learning approaches have gained widespread adoption in recent years: the generative adversarial network (GAN) first introduced in the work of Goodfellow et al. [9] and the restricted Boltzmann machine (RBM) invented by Smolensky [10] and further developed, among others, by Hinton et al. [11,12]. Therefore, it should not come as a surprise that all references mentioned above describe real-world applications of generative models based on either GAN or RBM approaches.

The arrival of a new computational paradigm of quantum computing and the progress achieved in developing quantum computing hardware prompted intensive research in exploring the capabilities of quantum machine learning models and, more specifically, quantum generative models that can be viewed as quantum counterparts of the classical GANs and RBMs. As we enter the NISQ (noisy intermediate scale quantum) era of quantum computing [13] and quantum supremacy being now an established experimental fact [14], there is a need to find examples of quantum advantage – real-world applications of quantum computing that achieve results beyond the capabilities of classical computers.

Quantum generative models have all the necessary qualities to establish quantum advantage on NISQ devices. Probably the most well-known example of such models is the quantum circuit Born machine (QCBM), a special type of parameterized quantum circuit (PQC). PQCs are usually composed of fixed 2-qubit quantum logic gates implementing entanglement (e.g. controlled NOT gates) and adjustable 1-qubit quantum logic gates (qubit rotations). A QCBM consists of several layers of adjustable and fixed gates followed by the measurement operators. The input is a quantum state where all qubits are initialized as $|0\rangle$ in the computational basis. The output is a bit string which is a sample from the probability distribution encoded in the final state constructed by the application of adjustable and fixed gates to the initial state.

Extensive literature dedicated to the QCBM as a model, with very high chances of demonstrating quantum advantage, is motivated by the following observations. First, QCBMs have strictly more expressive power than classical RBMs when only a

polynomial number of parameters is allowed (number of qubits in QCBM or number of visible activation units in RBM) [15]. Second, generation of an independent sample from the learned distribution can be done in a single run of the quantum circuit in the case of the QCBM – this compares favorably with up to 10^3–10^4 forward and backward passes through the network in the case of the RBM, which are sometimes needed to achieve the state of thermal equilibrium [1]. Since a single run of the relatively deep quantum circuit can be executed in ~10 μs (ignoring the read out and other overheads),[1] this points toward material quantum speed-up. Third, quantum generative models can be used to load data into the quantum state, thus facilitating realizations of many promising quantum algorithms [16].

However, the quantum advantage can only be achieved if we can find an efficient method of training the PQC models. Even some types of classical neural networks pose challenging training problems, not least GANs [18]. In some cases the differential learning of PQCs (a direct analogue of classical backpropagation of error with gradient descent technique) is a viable choice and produces acceptable results [19]. In other cases (for example, when the loss function is highly non-smooth) it may be more efficient to use the non-differentiable learning methods such as global optimization algorithms which are well suited for solving complex non-linear optimization problems that are NP-hard for classical computers: simulated annealing (SA) [20], genetic algorithm (GA) [21,22], Bayesian optimization (BO) [23], particle swarm optimization (PSO) [23,24].

In this paper we propose using GA as a non-differentiable learning algorithm of choice for training QCBMs. GA is, arguably, one of the most powerful evolutionary search heuristics. This power comes from the algorithm's ability to adapt to the characteristics of the specific optimization problem being solved. Due to this fact, GA can be viewed as a framework for solving complex non-linear optimization problems rather than a narrowly defined routine.

The paper is organized as follows. In Section 2 we describe the QCBM architecture and introduce the GA-based training procedure. The GA framework gives us full freedom in choosing the form of the loss function that ensures the best possible fit of the learned distribution encoded in the quantum state to the empirical distribution of the training dataset. In Section 3 we specify an equivalent classical counterpart: an RBM with the same number of activation units in the visible layer as the number of qubits in the QCBM and the same number of activation units in the hidden layer as the number of adjustable 1-qubit gates acting on every qubit in the QCBM. This ensures that the total number of weights in the RBM is equal to the total number of adjustable parameters (rotation angles) in the QCBM. In Section 4 we present the results of numerical experiments comparing the performance of QCBM trained with a GA against the performance of the equivalent RBM trained with a contrastive divergence algorithm. We also investigate the question of GA convergence as a function of a QCBM architecture and the algorithm's hyperparameters. Finally, in Section 5 we conclude and outline directions of further research.

2 Quantum circuit Born machine

A QCBM is a PQC where a layer of adjustable 1-qubit gates is followed by a layer of fixed 2-qubit gates. Such pattern can be repeated any number of times, building a progressively deeper circuit. The input is a quantum state where all qubits are initialized as $|0\rangle$ in the computational basis. The final layer consists of measurement operators producing a bitstring sample from the learned distribution. Therefore, to specify the QCBM architecture means to specify the number of layers, the type of adjustable gates, and the type of fixed gates for each layer. Since the theory of PQC is still being developed [25], we can rely on similarities and analogies between PQCs and classical neural networks in order to come up with some initial guesses about the possible QCBM architecture.

One can argue that a multi-layer perceptron (MLP), a simple feedforward neural network, is the closest classical neural network counterpart of the QCBM in terms of architecture, with the hidden layers of the MLP corresponding to the layers of adjustable and fixed quantum gates in the QCBM. Although a feedforward neural network with a single hidden layer and a continuous sigmoidal activation function [26] or, more generally, an appropriately smooth activation function [27] is capable of arbitrarily accurate approximation to an arbitrary function and its derivatives, it is a generally known practical observation [28] that no more than two hidden layers are needed to approximate an arbitrary function with relaxed requirements for the type of activation function.

Therefore, we suggest using two layers of fixed 2-qubit gates sandwiched between layers of adjustable 1-qubit gates as proposed in [29]. We are also interested in investigating the impact of stronger/weaker entanglement that can be achieved through different choices of 2-qubit gates. Our initial guess is that *CX* (controlled bit-flip) and *CZ* (controlled phase-flip) should result in the strongest effects, while controlled rotations by smaller rotation angles should lead to weaker effects. Appendix 5 visualizes qubit rotations with the help of the Bloch sphere and canonical representation of the qubit state.

Figure 1 displays a 12-qubit QCBM with two layers of controlled rotation gates ($V = R_\gamma(\theta)$, $\gamma = x, y, z$, $-\pi \le \theta < \pi$) where parameters γ and θ are fixed, and three layers of 1-qubit gates ($R_x(\theta)$, $R_z(\theta)$) with a total of seven adjustable gates per qubit (with hardware-efficient configuration of the middle rotation layer [30]). The circuit is wide enough and deep enough to learn a complex distribution of a continuous random variable while remaining implementable on the existing NISQ devices: the 12-digit binary representation of a continuous random variable provides sufficient precision and seven adjustable parameters (rotation angles) per qubit provide sufficient flexibility. At the same time the circuit is not too deep to be compromised by the gates' fidelity achievable in existing quantum hardware [17,31].

The chosen QCBM architecture is compatible with the limited connectivity we observe in the current generation of quantum processors. For example, the proposed circuit requires sequential qubit connectivity where qubit n is directly connected with qubits $n - 1$ and $n + 1$ but does not have to be directly connected with other qubits. This architecture can be supported by, e.g., IBM's Melbourne system [32] shown in Figure 2.

The 53-qubit Rochester device [32] shown in Figure 3 can also be used to implement the proposed QCBM architecture.

2.1 Training QCBM with genetic algorithm

Genetic algorithms are adaptive methods of searching a solution space by applying operators modeled after the natural genetic inheritance and simulating the Darwinian struggle for survival [33]. A GA performs a multi-directional search by maintaining a population of proposed solutions (chromosomes) for a given problem. Each solution is represented in a fixed alphabet with an established meaning (genes). The population undergoes a simulated evolution, with relatively good solutions producing offspring, which subsequently replace the worse ones. The estimate of the quality of a solution is based on an objective function (environment). GAs find applications in such diverse fields as quantitative finance (for solving portfolio opti-

[1] Taking into account that a 1-qubit gate time is ~10–2 μs and a 2-qubit gate time is ~10–1 μs for the superconducting qubits, the depth of the quantum circuit is limited by the gates' fidelity rather than by the decoherence time (the current state-of-the-art superconducting qubits have 1-qubit gate fidelity of up to 99.9% and 2-qubit gate fidelity of up to 99.4%, with decoherence time of up to 102 μs [17]).

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 1: QCBM(12, 7)

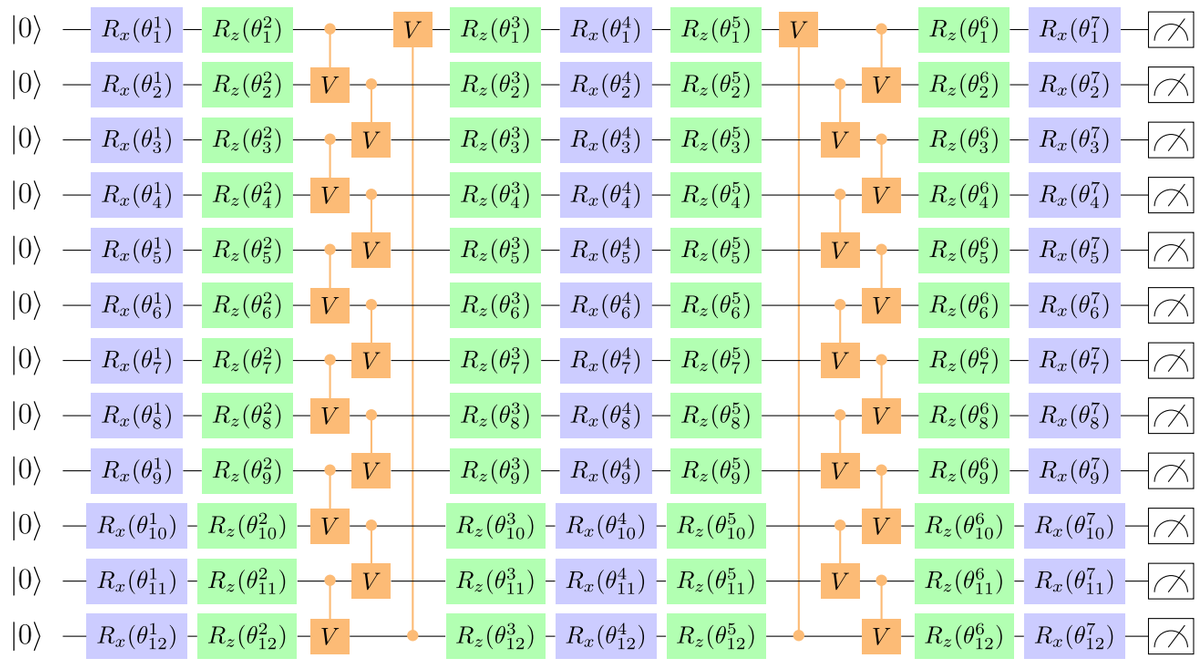
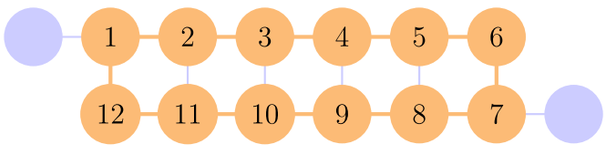


Figure 2: IBM's Melbourne system



mization problems [34]) and experiments with adiabatic quantum computing (as a classical benchmark [35]).

The simulation cycle is performed in three basic steps. During the selection step a new population is formed by stochastic sampling (with replacement). Then, some of the members of the newly selected populations recombine. Finally, all new individuals are re-evaluated. The mating process (recombination) is based on the application of two operators: mutation and crossover. Mutation introduces random variability into the population, and crossover exchanges random pieces of two chromosomes in the hope of propagating partial solutions.

Training of the QCBM specified in Figure 1 consists of finding an optimal configuration of the rotation angles θ_i^j, $i = 1, \ldots, 12$, $j = 1, \ldots, 7$ that would minimize a chosen objective function given a particular choice of the fixed 2-qubit gates. Since we only deal with 84 adjustable parameters (rather than tens of thousands), we do not need to implement the crossover mechanism and can rely on parameter mutation to achieve GA convergence to the minimum of the objective function. This significantly simplifies the algorithm.

Algorithm 1 outlines the proposed approach. However, before we provide a formal description of the algorithm, we have to specify the main individual components.

Solution. The solution is a 12×7 matrix of rotation angles:

$$\hat{\theta} = \begin{bmatrix} \theta_1^1 & \ldots & \theta_1^7 \\ \vdots & \ddots & \vdots \\ \theta_{12}^1 & \ldots & \theta_{12}^7 \end{bmatrix}. \tag{1}$$

In GA language, the matrix $\hat{\theta}$ plays the role of a chromosome and its components θ_i^j play the roles of individual genes.

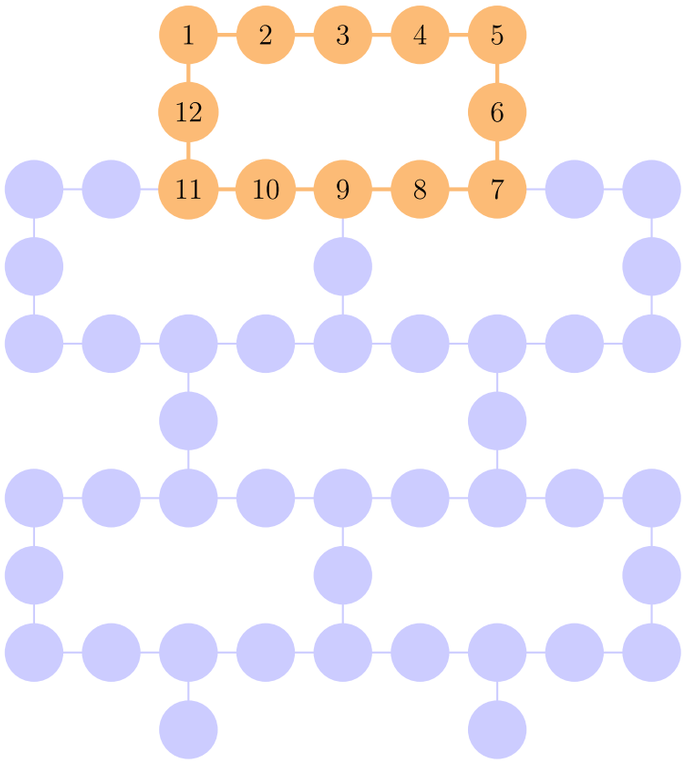
Mutation. The genes can mutate from generation to generation. The mutation rate can be either constant or time-dependent. For example, the mutation rate can start at some large value and then decrease exponentially such that it halves after each κ generations. In Algorithm 1 we adopt the following mutation dynamics:

- A rotation angle (gene) can mutate to any of the allowed discrete values with equal probability.
- Mutation is controlled by a single global parameter, $0 < \alpha \leq 1$, which can be either constant or exponentially decreasing at some fixed rate $\beta \geq 0$.
- Mutations happen independently for each column in $\hat{\theta}$.
- For each column in $\hat{\theta}$, at each generation, a single rotation angle mutation happens with probability α. All rotation angles are equally likely to mutate. After that, one more mutation can happen with probability $\alpha/2$. Again, all rotation angles are equally likely to mutate. This ensures that we can have scenarios where two rotation angles within the same column can mutate simultaneously.

Search space. Rotation angles θ_i^j are defined on the interval $-\pi \leq \theta_i^j < \pi$. This interval can be split into 2^m equal subintervals. Then the possible discrete values for θ_i^j are $-\pi + n\pi/2^{m-1}$, $n = 0, \ldots, 2^m - 1$. A rotation angle can mutate into any of these values. The search space can quickly become enormous even for the relatively small values of m. For example, for $m = 7$ we have 128 possible values for each rotation angle,

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 3: IBM's Rochester system



making the total number of possible configurations $128^{84} \simeq 10^{177}$. The GA can only explore a tiny fraction of the search space. But due to the GA's ability to propagate best solutions and to avoid being trapped in local minima, the algorithm can achieve reasonably fast convergence to the solution in the vicinity of the global minimum.

Objective function. An objective function is a measure of how far the distribution of generated samples is from the distribution of original samples provided by the training dataset. Let $\mathbf{u} = (u_1, \ldots, u_K)$ be a vector of samples from the training dataset and $\mathbf{v}(\hat{\theta}) = (v_1(\hat{\theta}), \ldots, v_K(\hat{\theta}))$ be a vector of samples from the QCBM-generated dataset that corresponds to a particular configuration of rotation angles $\hat{\theta}$. Let us order these samples from the smallest to the largest with any suitable *sort*() function:

$$\overline{\mathbf{u}} = sort(\mathbf{u}), \quad \overline{\mathbf{v}}(\hat{\theta}) = sort(\mathbf{v}(\hat{\theta})).$$

Then the objective function $F(\hat{\theta})$ can be defined as follows:

$$F(\hat{\theta}) = \sum_{k=1}^{K} (\overline{u}_k - \overline{v}_k(\hat{\theta}))^2. \tag{2}$$

3 Restricted Boltzmann machine

An RBM is a two-layer neural network with stochastic activation units. "Restricted" in RBM refers to a constraint imposed on the network architecture: there are no connections between the units within the same layer. The RBM layer which is exposed to the training dataset is called the visible layer. Inputs from the visible layer flow through the network (forward pass) to the hidden layer, where they are aggregated and added

Algorithm 1: Genetic algorithm

Result: An optimal configuration of adjustable QCBM parameters $\hat{\theta}^*$ that minimizes the value of the objective function.
Input: $\mathbf{u} = (u_1, \ldots, u_K)$, vector of samples (training dataset).
Input: L, number of iterations (generations).
Input: N, number of solutions in each generation.
Input: M, number of best solutions in the given generation that are chosen for further mutation.
N should be an integer multiple of M: $N = DM, D \in \mathbb{N}$.
Input: α and β, mutation parameters.
Input: m, search space parameter. The possible values of rotation angles are:

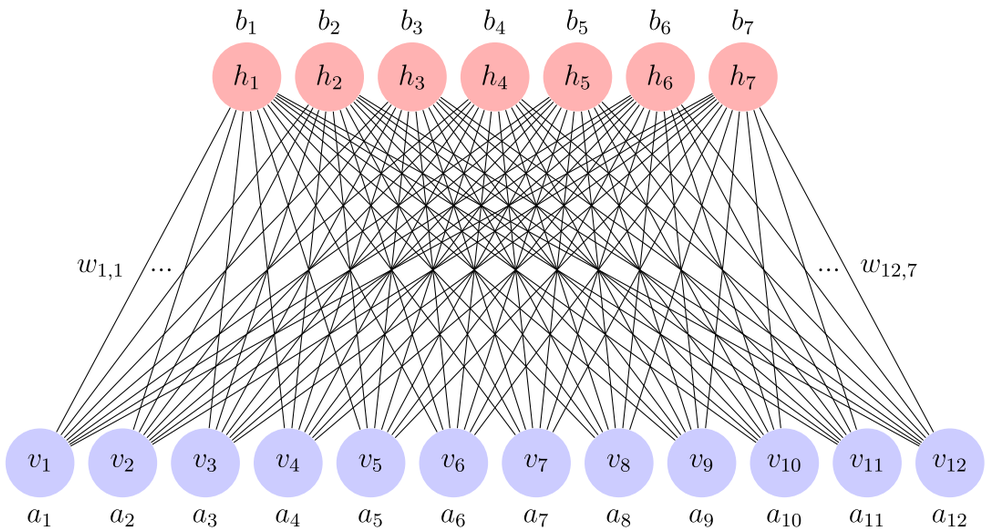
$$\left\{-\pi, -\pi + \pi/2^{m-1}, -\pi + 2\pi/2^{m-1}, \ldots, \pi\left(1 - \frac{1}{2^{m-1}}\right)\right\}.$$

Initialization and evaluation of the first generation of solutions:
for $n = 1, \ldots, N$ **do**
  Generation of configuration $\hat{\theta}(0; n)$ by randomly drawing each rotation angle $\theta_i^j(0; n)$ from the uniform distribution on the set of possible values of rotation angles given by m.
  for $k = 1, \ldots, K$ **do**
    Running of the quantum circuit with configuration $\hat{\theta}(0; n)$ and generation of new sample $v_k(\hat{\theta}(0; n))$.
  end
  Evaluation of the objective function $F(\hat{\theta}(0; n))$.
end
Ordering of solutions from best (minimum of objective function) to worst (maximum of objective function).
$\hat{\theta}^* \leftarrow$ configuration corresponding to the minimum of objective function.

Iterations:
for $l = 1, \ldots, L$ **do**
  $\alpha \leftarrow \alpha \exp(-\beta)$.
  Selection of M best solutions from generation $l - 1$ and generation of N new solutions $\hat{\theta}(l; n)$, $n = 1, \ldots, N$, by mutating the rotation angles using the updated mutation rate α. Each of the M best solutions is used to produce D new solutions.
  for $n = 1, \ldots, N$ **do**
    for $k = 1, \ldots, K$ **do**
      Running of the quantum circuit with configuration $\hat{\theta}(l; n)$ and generation of new sample $v_k(\hat{\theta}(l; n))$.
    end
    Evaluation of the objective function $F(\hat{\theta}(l; n))$.
  end
  Ordering of solutions from best (minimum of objective function) to worst (maximum of objective function).
  $\hat{\theta}^*(l) \leftarrow$ configuration corresponding to the minimum of objective function (lth generation).
  if $F(\hat{\theta}^*(l)) < F(\hat{\theta}^*)$ **then**
    $\hat{\theta}^* \leftarrow \hat{\theta}^*(l)$
  end
end

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 4: RBM(12, 7)



to the hidden layer biases. The hidden layer sigmoid activation function converts aggregated inputs into probabilities. Each hidden unit then "fires" randomly – its output is a Bernoulli random variable: "1" is generated with probability p, which is equal to the sigmoid activation function value, and "0" is generated with probability $1 - p$.

The outputs from the hidden layer then flow back (backward pass) to the visible layer, where they are aggregated and added to the visible layer biases. Similar to the hidden layer, the visible layer sigmoid activation function translates aggregated inputs into probabilities. However, the visible layer does not necessarily convert probabilities into random variables – some of the RBM implementations output visible layer probabilities rather than random variables.

The network learns the joint distribution of the configurations of visible and hidden activation units by trying to reconstruct the inputs from the training dataset (visible unit values) by finding an optimal set of the network weights and biases. Note that the same network weights are used for the forward and backward passes. We provide below a sketch of the training algorithm based on the stochastic gradient ascent approach that maximizes the log likelihood of the training data.

Figure 4 shows an the RBM with 12 stochastic binary visible activation units and seven stochastic binary hidden activation units (a_i, b_j and w_{ij}, $i = 1, \ldots, 12$, $j = 1, \ldots, 7$, denote, respectively, the biases for the visible and hidden layers and the network weights). This network architecture makes the RBM equivalent to the QCBM described in Section 2 in the sense that both generative models have the same number of adjustable parameters (the number of RBM weights is equal to the number of adjustable rotation angles in the QCBM) and the number of visible activation units is equal to the number of quantum registers. The latter ensures that both generative models can learn empirical distribution of continuous random variables with the same precision (12-digit binary representation).

3.1 Training RBM with contrastive divergence algorithm

The network is trained through the updates of weights and biases which increase the log probability of a training vector and are given by the following expressions [1,36,37]:

$$\Delta w_{ij} = \eta \left(\langle v_i h_j \rangle_{data} - \langle v_i h_j \rangle_{model} \right),$$

$$\Delta a_i = \eta \left(\langle v_i \rangle_{data} - \langle v_i \rangle_{model} \right), \quad \Delta b_j = \eta \left(\langle h_j \rangle_{data} - \langle h_j \rangle_{model} \right),$$

where $\langle \ldots \rangle$ denote expectations under the distribution specified by the subscript and η is the chosen learning rate. Expectations $\langle \ldots \rangle_{data}$ can be calculated directly from the training dataset while getting unbiased samples of $\langle \ldots \rangle_{model}$ requires performing alternating sampling from the model Boltzmann distribution for a long time (needed to achieve the state of thermal equilibrium), starting from some randomly initialized state. However, the k-step contrastive divergence method [1,36,37] can be used to approximate $\langle \ldots \rangle_{model}$ with another, easier to calculate expectation as shown in Algorithm 2.

Sampling from the trained RBM consists of three steps: (i) generation of a random input, (ii) performing T forward and backward passes through the network (alternating Boltzmann sampling), and (iii) taking the generated visible unit values after the last backward pass as an output. An RBM trained with the bottleneck hidden layer structure works as an autoencoder that tries to reconstruct the input. In order to generate an independent sample it is necessary to achieve the state of thermal equilibrium where practically all information about the input is lost. Therefore, the *thermalization* parameter T should be large enough to ensure the independence of the generated samples [1].

4 Numerical experiments

Both QCBM and RBM operate on binary variables. Therefore, we need to design a method of converting real-valued samples into binary features that can be taken by the network as an input for training and a method of converting the generated binary network output (sampling) into real-valued variables. Such a method can be realized as a two-step routine:

1. Conversion of real-valued samples into the corresponding integer samples.
2. Conversion of integer samples into the corresponding binary features.

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Algorithm 2: k-Step contrastive divergence Weights and biases updates

Result: Input: training minibatch S
Input: model parameters $a_i, b_j, w_{ij}, i = 1, \ldots, N, j = 1, \ldots, M$ (before update)
Initialization: $\forall i, j : \Delta w_{ij} = \Delta a_i = \Delta b_j = 0$
for $\mathbf{v} \in S$ **do**
  $\mathbf{v}^{(0)} \leftarrow \mathbf{v}$
  for $t = 0, \ldots, k-1$ **do**
    for $j = 1, \ldots, M$ **do**
      sample Bernoulli random variable $h_j^{(t)} \sim p(h_j|\mathbf{v}^{(t)})$
    end
    for $i = 1, \ldots, N$ **do**
      sample Bernoulli random variable $v_i^{(t+1)} \sim p(v_i|\mathbf{h}^{(t)})$
    end
  end
  for $i = 1, \ldots, N, j = 1, \ldots, M$ **do**
    $\Delta w_{ij} \leftarrow \Delta w_{ij} + \eta \left(p(h_j = 1|\mathbf{v}^{(0)}) v_i^{(0)} - p(h_j = 1|\mathbf{v}^{(k)}) v_i^{(k)} \right)$
  end
  for $i = 1, \ldots, N$ **do**
    $\Delta a_i \leftarrow \Delta a_i + \eta \left(v_i^{(0)} - v_i^{(k)} \right)$
  end
  for $j = 1, \ldots, M$ **do**
    $\Delta b_j \leftarrow \Delta b_j + \eta \left(p(h_j = 1|\mathbf{v}^{(0)}) - p(h_j = 1|\mathbf{v}^{(k)}) \right)$
  end
end

Given the generated binary output, the same routine can be used in the reverse mode to produce real-valued samples:

1. Conversion of the generated binary network output into integer samples.
2. Conversion of integer samples into the corresponding real-valued samples.

The following Algorithms 3 and 4 [1] describe transformations of real-valued samples into M-digit binary features and then back into real-valued samples. We test the performance of the QCBM and RBM on two datasets.

- **Dataset 1.** A specially constructed distribution of a continuous random variable with a highly spiky probability density function (PDF) modeled as a mixture of normal distributions. The dataset consists of 5,000 generated samples from a mixture of four normal distributions with the following mean values, standard deviations, and weights:

Mean	Standard deviation	Weight
– 3	0.3	0.1
– 1	0.3	0.2
1	0.3	0.3
3	0.3	0.4

Algorithm 3: Real-valued to integer to binary transformation (training phase)

Result: Conversion of real-valued dataset into M-digit binary features
Input: $X_{real}^{(n)}(l)$ – a real-valued dataset: $l = 1, \ldots, N_{samples}, n = 1, \ldots, N_{variables}$
for $n = 1, \ldots, N_{variables}$ **do**
  $X_{min}^{(n)} \leftarrow \min_{\{l\}}(X_{real}^{(n)}) - \epsilon_{min}^{(n)}, \quad \epsilon_{min}^{(n)} \geq 0$
  $X_{max}^{(n)} \leftarrow \max_{\{l\}}(X_{real}^{(n)}) + \epsilon_{max}^{(n)}, \quad \epsilon_{max}^{(n)} \geq 0$
  for $l = 1, \ldots, N_{samples}$ **do**
    $X_{integer}^{(n)}(l) \leftarrow \text{int}\left(\left(2^M - 1\right) \times \left(X_{real}^{(n)}(l) - X_{min}^{(n)} \right) / \left(X_{max}^{(n)} - X_{min}^{(n)} \right) \right)$
    $X_{binary}^{(n)}(l) \leftarrow \text{binarize}\left(X_{integer}^{(n)}(l) \right)$
  end
end
Each data sample is represented by an M-digit binary number with every digit becoming a separate feature. The total number of features is $M \times N_{variables}$.

Algorithm 4: Binary to integer to real-valued transformation (sampling phase)

Result: Conversion of the generated M-digit binary sample into real-valued sample
Input: $\hat{X}^{(n)}[m]$ – generated M-digit binary sample: $m = 0, \ldots, M-1$, $n = 1, \ldots, N_{variables}$
for $n = 1, \ldots, N_{variables}$ **do**
  $\hat{X}_{integer}^{(n)} \leftarrow 0$
  for $m = 0, \ldots, M-1$ **do**
    $\hat{X}_{integer}^{(n)} \leftarrow \hat{X}_{integer}^{(n)} + 2^m \times \hat{X}^{(n)}[M-1-m]$
  end
  $\hat{X}_{real}^{(n)} \leftarrow X_{min}^{(n)} + \hat{X}_{integer}^{(n)} \times \left(X_{max}^{(n)} - X_{min}^{(n)} \right) / \left(2^M - 1\right)$
end

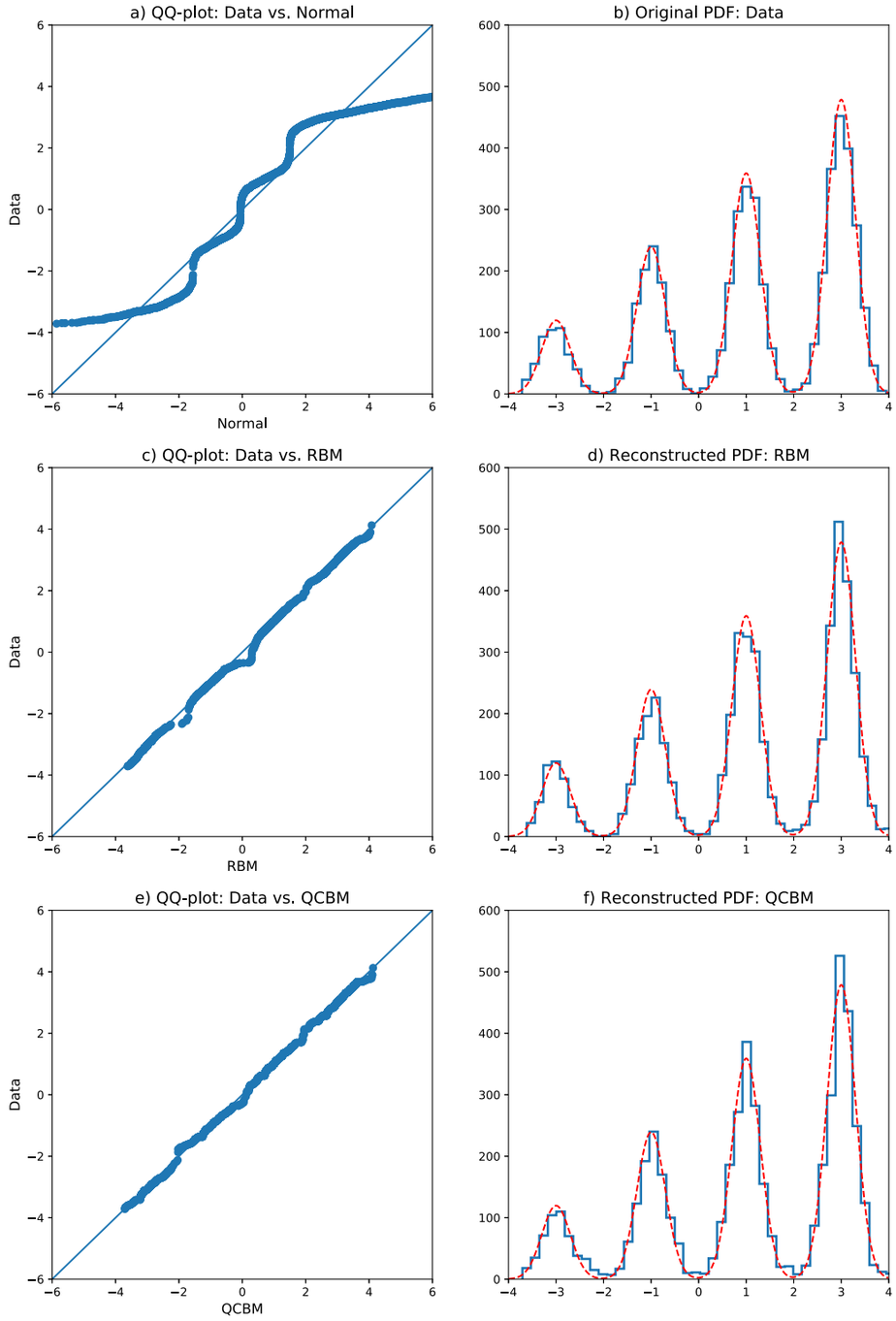
- **Dataset 2.** A heavy-tailed distribution of spot FX daily log-returns for the USD–JPY currency pair observed between 2001 and 2017 (and, therefore, covering several periods of market distress). Given a time series of daily observations of the foreign exchange rates $S_0, S_1, \ldots, S_K$, the daily log-returns are calculated as $\ln(S_k/S_{k-1}), k = 1, \ldots, K$. The dataset consists of 4,382 samples.

In both cases we convert continuous samples into the corresponding 12-digit binary representation as per Algorithm 3. Once the networks are trained (QCBM with Algorithm 1 and RBM with Algorithm 2), we generate new samples: 5,000 new samples for dataset 1 and 4,382 new samples for dataset 2. This allows us to visualize the quality of the generated samples (once they are converted into the corresponding continuous representation as per Algorithm 4) by producing the PDF and QQ plots[2] as shown in Figures 5 and 6, which display sample simulation results for the fully trained models.

[2] The quantile–quantile (QQ) plot is a scatterplot created by plotting two sets of quantiles against one another. If both sets of quantiles come from the same distribution, we should see the points lying close to a straight line.

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 5: Mixture of normal distributions

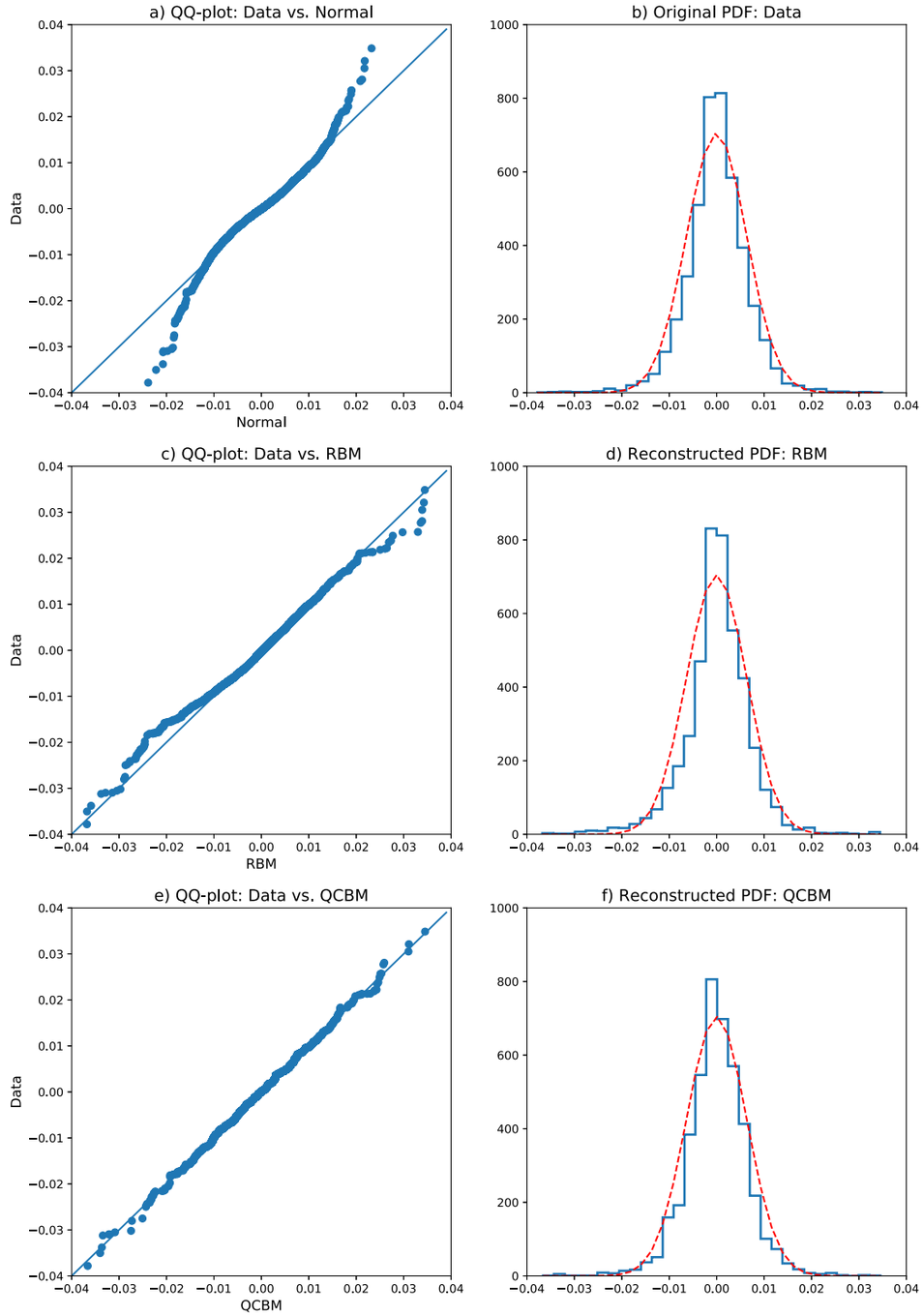


We can see that both QCBM(12, 7) and RBM(12, 7) can successfully learn complex empirical distributions (light-tailed with spiky PDF in the case of dataset 1 and heavy-tailed in the case of dataset 2). We have chosen *CX* for the fixed gates in QCBM and used the open-source Qiskit package [38] to simulate the quantum parts of the training and sampling algorithms.

The following sets of hyperparameters were used to train the models.

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 6: Distribution of USD–JPY spot FX daily log-returns (2001–2017)



Genetic algorithm for training QCBM (Algorithm 1)

$N = 1{,}000$ – number of new solutions in each generation

$M = 25$ – number of top solutions chosen for further mutation from each generation

$m = 7$ – possible number of rotation angles is $2^7 = 128$

$\alpha = 1.0$ – the largest possible initial mutation rate

$\beta = 0.013863$ – ensures that mutation rate halves after each $\kappa = 50$ generations

$L = 200$ – number of generations (iterations)

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Contrastive divergence algorithm for training RBM
(BernoulliRBM from the open-source Scikit-learn package [39])
n_components = 7 – number of hidden activation units for RBM(12, 7)
learning_rate = 0.0005 – learning rate η in Algorithm 2
batch_size = 10 – size of the training minibatches S in Algorithm 2
n_iter = 40,000 – number of iterations

Although the optical inspection of PDF and QQ plots in Figures 5 and 6 suggests that both QCBM and RBM are doing a good job in generating high-quality samples from the learned empirical distributions encoded in model parameters, we would like to have a more objective measure of the model performance. This is especially important since we deal with the generative models and, therefore, we cannot base our conclusion on a single model run.

Running the quantum circuit multiple times for a particular configuration of model parameters (e.g., an optimal set of rotation angles found with the help of a GA) results in the distribution of objective function values. This gives us an idea of what metrics can be used to measure the performance of the QCBM and RBM. The objective function (2) we used for training the QCBM can be calculated on the samples generated by the RBM. In other words, we can compare the performance of the QCBM and RBM by comparing the distribution of the objective function values calculated for the samples generated by these models.

Table 1 shows the mean and standard deviation of the objective function values calculated for 100 runs of QCBM(12, 7) and RBM(12, 7). Each run generated 5,000 samples from the learned empirical distribution (models were trained on dataset 1 which consists of 5,000 samples from the mixture of four normal distributions).

It is clear from Table 1 that QCBM(12, 7) with a weakly optimized set of hyperparameters performs better than RBM(12, 7) trained with equally weakly optimized hyperparameters (small learning rate combined with large number of iterations and a small size of the minibatches [36]). Although it cannot be seen as evidence of the quantum advantage, this result nevertheless points to a promising area of further research.

Next, we would like to explore the GA behavior for various model configurations. In particular, it is interesting to investigate the algorithm convergence for different types of fixed gates, not just CX, and for different choices of the mutation rate. The charts in Figure 7 confirm our intuition about CX being the best choice of the fixed gate type given the configuration of 1-qubit gates (Figure 1) and exponentially decreasing mutation rate performing better than the constant mutation rates. Here, we continue working with dataset 1.

As we can see in Figure 1, the fixed gates are flanked by 1-qubit gates performing rotations around the z-axis. Therefore, adding another rotation around the z-axis by $\theta = \pi$ ($CZ \equiv CR_z(\pi)$) may not offer the same flexibility as rotation around the x-axis by $\theta = \pi$ ($CX \equiv CR_x(\pi)$). Controlled rotations around the z-axis by the angle $\theta < \pi$ are likely to perform even worse. This is exactly what we see in Figure 7 (left chart) for three different types of fixed gates: CX, CZ, and $CR_z(\pi/4)$.

Our intuition about the optimal choice of mutation rate suggests that it should be productive to start the algorithm with a really large mutation rate in order to explore the search space as broadly as possible (the "exploration" phase). Then, as the algorithm finds progressively better solutions, it should be useful to reduce the mutation rate in order to perform a more detailed search in the vicinity of the best solutions found so far (the "exploitation" phase). As the algorithm converges, we may want to perform more and more refined searches by only mutating one or two parameters. Figure 7 (right chart) shows that this is indeed the case. Here, the maximum value of the mutation rate is $\alpha = 1.0$ and the minimum value is $\alpha = 0.0625$ – the value reached after $L = 200$ algorithm iterations when the algorithm is run with the initial value of mutation rate $\alpha = 1.0$ and exponential decay factor $\beta = 0.013863$.

Finally, we need to investigate algorithm convergence as a function of rotation angle discretization scheme. In principle, an arbitrary rotation poses a problem as it must be approximated by a sequence of discrete gates, because only discrete sets of gates can be implemented fault-tolerantly [40]. Since a GA operates on a discrete set of rotation angles, we face a trade-off between higher accuracy achieved through a finer discretization scheme and implementation efficiency in the case of a less granular set of rotation angles. Additionally, all rotation gates can be executed with finite precision and the discretization scheme should take this into account. Hence, in order to facilitate efficient implementation of the rotation gates $R_x(\theta)$ and $R_z(\theta)$, the GA operates on rotation angles θ that take discrete values $-\pi + n\pi/2^{m-1}$, $n = 0, \ldots, 2^m$, thus splitting the $[-\pi, \pi]$ interval into 2^m equal subintervals.

Therefore, we must answer the question of GA convergence for various values of m. Figure 8 shows the minimum values of the objective function (2) as a function of the number of algorithm iterations for three different values of m:

- $m = 3$, rotation angle step $\Delta\theta = \pi/4$;
- $m = 5$, rotation angle step $\Delta\theta = \pi/16$;
- $m = 7$, rotation angle step $\Delta\theta = \pi/64$.

We can see that the GA performance improves only marginally for $m > 5$. This is good news, suggesting that it may be sufficient to operate with rotation angle step $\Delta\theta = \pi/16$ to achieve the desired precision in learning the target distribution. A possible explanation for why the relatively wide rotation angle step may be sufficient can be derived from analogy with classical neural networks. A feedforward neural network may benefit more from adding an extra hidden layer than from doubling the number of activation units in existing hidden layers. In other words, increasing the network depth may be relatively more important than increasing the network width. When it comes to the proposed QCBM architecture (Figure 1), it seems that the QCBM is deep enough to learn complex distributions with the limited number of available rotation angles.

Table 1: Objective function statistics for the models trained on dataset 1

Model	Mean	Standard deviation
QCBM(12, 7)	30.5	23.6
RBM(12, 7)	39.6	30.8

5 Conclusions and further research

The proposed non-differentiable learning of the QCBM with a GA proves to be a viable approach to the training of parametrized quantum circuits. The QCBM trained with a GA performs at least as well as an equivalent classical neural network (RBM). The performance of the QCBM and its classical counterpart were tested on two different datasets: (i) heavy-tailed distributed samples derived from the financial time series and (ii) light-tailed distributed samples drawn from the specially constructed distribution with spiky PDF. In both cases, the QCBM demonstrated an ability to learn the empirical distribution and generate new synthetic samples that have the same statistical properties as the original one, as can be seen in the PDF and QQ plots.

The GA convergence was analysed for different sets of hyperparameters. The best results were achieved for the CX fixed gates and exponentially decreasing mutation rate (starting from the maximum value of the mutation rate and setting the decay rate at a reasonably small value). More importantly, we see that more granular

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure 7: Left chart: GA convergence as a function of fixed gate type. Right chart: GA convergence as a function of mutation rate for *CX* fixed gates. Dots indicate mean values and error bars indicate the 10th and 90th percentiles. GA parameters: *N* = 1, 000, *M* = 25, *m* = 7. Number of GA runs: 20

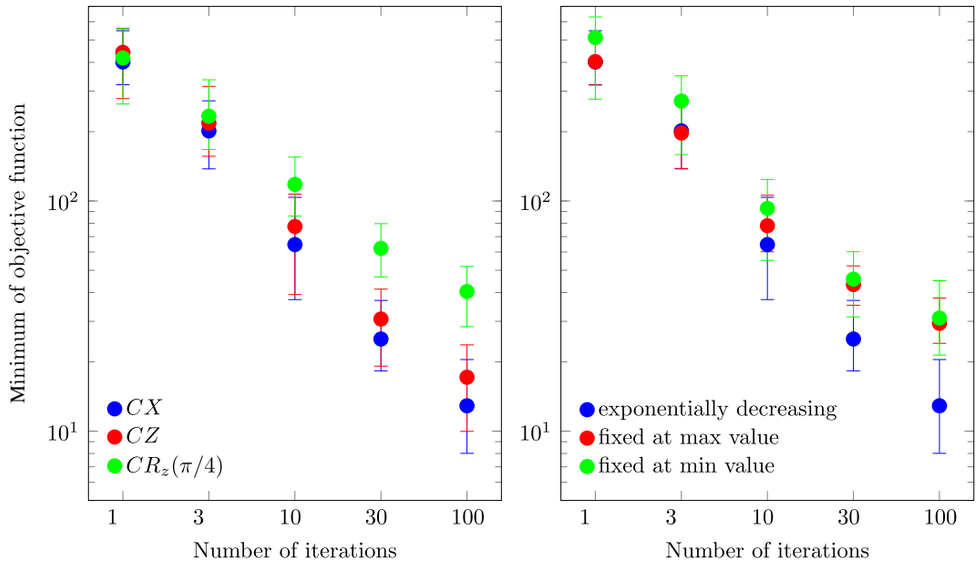
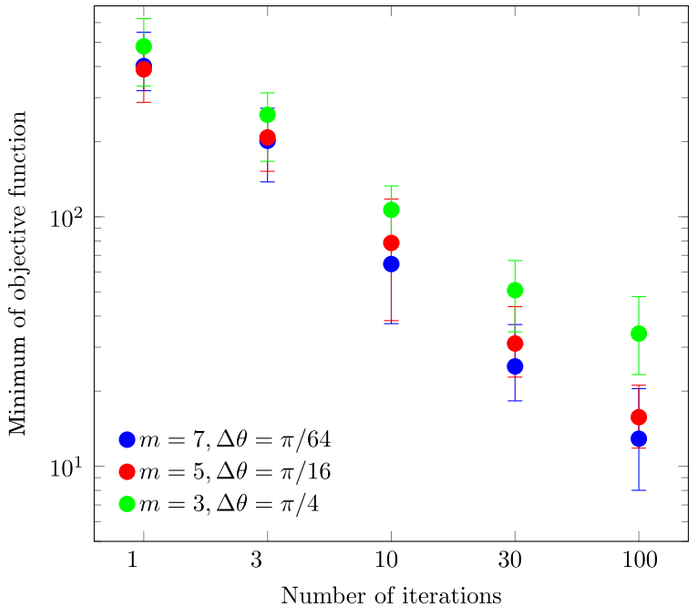


Figure 8: GA convergence as a function of rotation angle discretization scheme for *CX* fixed gates and exponentially decreasing mutation rate. Dots indicate mean values and error bars indicate the 10th and 90th percentiles. GA parameters: *N* = 1, 000, *M* = 25, *α* = 1.0, *β* = 0.013863. Number of GA runs: 20



rotation angle discretization schemes provide progressively less incremental value beyond some point. This means that for many practical purposes, it is sufficient to implement rotations with the step $\Delta\theta = \pi/16$ in order to encode the target distribution with the desired accuracy for deep enough QCBM architectures (at least two layers of fixed 2-qubit gates). Since qubit rotations on NISQ devices can be implemented with finite precision, this ensures that QCBMs can be used productively for many real-world use cases.

The QCBM runs were performed on the Qiskit simulator [38]. Testing of the proposed approach on actual quantum processors should provide additional insights and suggest directions of further research. There are indications that a tangible quantum advantage over comparable classical neural networks can be demonstrated, either in terms of the quality of generated samples (expressive power of the model) or in terms of the speed-up (the time needed to generate an independent sample from the learned distribution).

Acknowledgments

The author would like to thank Marco Paini, Stefan Woerner, and the anonymous reviewer for valuable comments that greatly improved the manuscript.

Disclaimer and Declaration of Interest

The author reports no conflict of interest. The author alone is responsible for the content and writing of the paper. The opinions expressed are those of the author. All figures are based on the author's own calculations.

Alexei Kondratyev is a Visiting Professor at Department of Mathematics, Imperial College London. He has over 20 years of quant finance industry experience and has been recognized as Quant of the Year (2019) by Risk magazine for the research on application of machine learning techniques to the risk factor analysis and portfolio optimization. He holds an MSc in Theoretical Physics from Taras Shevchenko National University of Kiev and a PhD in Mathematical Physics from the Institute for Mathematics, National Academy of Sciences of Ukraine.

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

References

[1] Kondratyev, A. and Schwarz, C. 2020. The market generator. *Risk* 33(Feb). https://www.risk.net/cutting-edge/7401191/the-market-generator.
[2] Bühler, H., Horvath, B., Lyons, T., Perez Arribaz, I., and Wood, B. 2020. A data-driven market simulator of financial time series for small data environments. arXiv preprint arXiv:2006.14498.
[3] Hyland, S. L., Esteban, C., and Rätsch, G. 2017. Real-valued (medical) time series generation with recurrent conditional GANs. arXiv preprint arXiv:1706.02633.
[4] Kondratyev, A., Schwarz, C., and Horvath, B. 2020. The data anonymiser. *Risk* 33(Sept). https://www.risk.net/cutting-edge/banking/7669111/the-data- anonymiser.
[5] Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. 2017. Unpaired image-to-image translation using cycle-consistent adversarial networks. arXiv preprint arXiv:1703.10593.
[6] Oh, S., Jung, Y., Kim, S., Lee, I., and Kang, N. 2019. Deep generative design: Integration of topology optimization and generative models. arXiv preprint arXiv:1903.01548.
[7] Yoshioka, N., Akagi, Y., and Katsura, H. 2018. Transforming generalized Ising model into Boltzmann machine. arXiv preprint arXiv:1812.05269.
[8] Melko, R. G., Carleo, G., Carrasquilla, J., and Ignacio Cirac, J. 2019. Restricted Boltzmann machines in quantum physics. *Nature Physics* 15, 887–892.
[9] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S. et al. 2014. Generative adversarial nets. arXiv preprint arXiv:1406.2661.
[10] Smolensky, P. 1986. Information processing in dynamical systems: Foundations of harmony theory. In Rumelhart, D. E. and McClelland, J. L. (eds), *Parallel Distributed Processing Explorations in the Microstructure of Cognition*, Vol 1: *Foundations*. Cambridge, MA: MIT Press.
[11] Hinton, G. and Salakhutdinov, R. 2006. Reducing the dimensionality of data with neural networks. *Science* 313, 504–507.
[12] Salakhutdinov, R., Mnih, A., and Hinton, G. 2007. Restricted Boltzmann machines for collaborative filtering. In: *Proceedings of the 24th International Conference on Machine Learning*, Corvallis, OR.
[13] Preskill, J. 2018. Quantum computing in the NISQ era and beyond. arXiv preprint arXiv:1801.00862.
[14] Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R. et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574, 505–510.
[15] Du, Y., Hsieh, M.-H., Li, T., and Tao, D. 2018. The expressive power of parameterized quantum circuits. arXiv preprint arXiv:1801.07686.
[16] Zoufal, C., Lucchi, A., and Woerner, S. 2019. Quantum generative adversarial networks for learning and loading random distributions. arXiv preprint arXiv:1904.00043.
[17] Kjaergaard, M., Schwartz, M. E., Braumüuller, J., Krantz, P., Wang, J. I-J., Gustavsson, S., and Oliver, W. D. 2019. Superconducting qubits: Current state of play. arXiv preprint arXiv:1905.13641.
[18] Arora, S. and Zhang, Y. 2017. Do GANs actually learn the distribution? An empirical study. arXiv preprint arXiv:1706.08224.
[19] Liu, J.-G. and Wang, L. 2018. Differentiable learning of Quantum Circuit Born Machine. arXiv preprint arXiv:1804.04168.
[20] Zhou, X., Li, S., and Feng, Y. 2019. Quantum circuit transformation based on simulated annealing and heuristic search. arXiv preprint arXiv:1908.08853.
[21] Lamata, L., Alvarez-Rodriguez, U., Martín-Guerrero, J. D., Sanz, M., and Solano, E. 2018. Quantum autoencoders via quantum adders with genetic algorithms. *Quantum Science and Technology* 4(1), 014007.
[22] Robert, A., Barkoutsos, P. Kl., Woerner, S., and Tavernelli, I. 2019. Resource-efficient quantum algorithm for protein folding. arXiv preprint arXiv:1908.02163.
[23] Zhu, D., Linke, N. M., Benedetti, M., Landsman, K. A., Nguyen, N. H., Alderete, C. H. et al. 2019. Training of quantum circuits on a hybrid quantum computer. *Science Advances* 5, eaaw9918.
[24] Benedetti, M., Garcia-Pintos, D., Perdomo, O., Leyton-Ortega, V., Nam, Y., and Perdomo-Ortiz, A. 2018. A generative modeling approach for benchmarking and training shallow quantum circuits. arXiv preprint arXiv:1801.07686.
[25] Benedetti, M., Lloyd, E., Sack, S., and Fiorentini, M. 2019. Parametrized quantum circuits as machine learning models. *Quantum Science and Technology*4, 043001.
[26] Cybenko, G. 1989. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2, 303–314.
[27] Hornik, K., Stinchcombe, M., and White, H. 1990. Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks. *Neural Networks* 3(5), 551–560.
[28] Amilon, H. 2003. A neural network versus Black–Scholes: A comparison of pricing and hedging performances. *Journal of Forecasting* 22(4), 317–335.
[29] Luo, X.-Z., Liu, J.-G., Zhang, P., and Wang, L. 2019. Yao.jl: Extensible, efficient framework for quantum algorithm design. arXiv preprint arXiv:1912.10877.
[30] Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J. M., and Gambetta, J. M. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* 549, 242–246.
[31] Bruzewicz, C. D., Chiaverini, J., McConnell, R., and Sage, J. M. 2019. Trapped-ion quantum computing: Progress and challenges. arXiv preprint arXiv:1904.04178.
[32] McClure, D. and Gambetta, J. 2019. Quantum computation center opens. https://www.ibm.com/blogs/research/2019/09/quantum-computation-center/.
[33] Mitchell, M. 1998. *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press.
[34] Kondratyev, A. and Giorgidze, G. 2017. Evolutionary algos for optimising MVA. *Risk* 30(Dec). https://www.risk.net/cutting-edge/banking/5374321/evolutionary-algos-for-optimising-mva.
[35] Venturelli, D. and Kondratyev, A. 2019. Reverse quantum annealing approach to portfolio optimization problems. *Quantum Machine Intelligence* 1, 17–30.
[36] Hinton, G. 2010. *A Practical Guide to Training Restricted Boltzmann Machines*. UTML TR 2010-003. Toronto: Department of Computer Science, University of Toronto.
[37] Fischer, A. and Igel, C. 2014. Training restricted Boltzmann machines: An introduction. *Pattern Recognition* 47(1), 25–39.
[38] Abraham, H., Akhalwaya, I. Y., Aleksandrowicz, G., Alexander, T., Alexandrowics, G., Arbel, E. et al. 2019. *Qiskit: An open-source framework for quantum computing*.
[39] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12(85), 2825–2830.
[40] Kudrow, D., Bier, K., Deng, Z., Franklin, D., Tomita, Y., Brown, K. R., and Chong, F. T. 2013. Quantum rotations: A case study in static and dynamic machine-code generation for quantum computers. In: *Proceedings of the International Symposium on Computer Architecture (ISCA-40)*, Tel Aviv, Israel.

Appendix A: The Bloch sphere and quantum gates

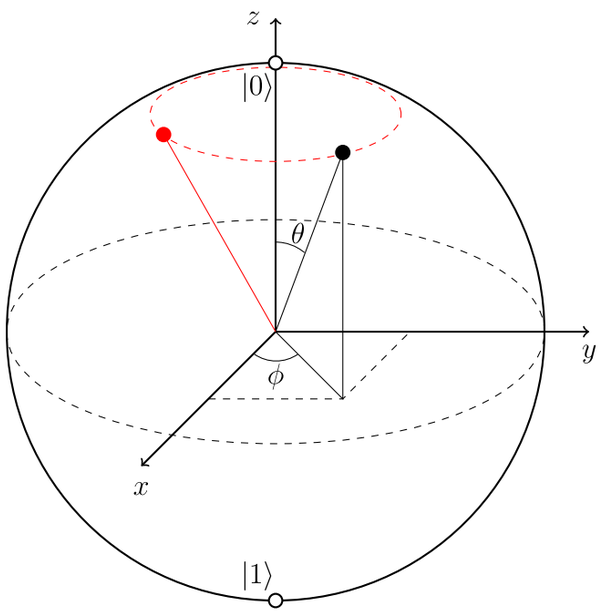
The Bloch sphere (Figure A1) visualizes the canonical representation of the qubit state – a unit vector in the two-dimensional complex vector space:

$$|\psi\rangle = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) \\ e^{i\phi}\sin\left(\frac{\theta}{2}\right) \end{bmatrix}. \tag{3}$$

Angles $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi]$ uniquely determine the position of the qubit on the unit sphere. Mathematically, a single qubit gate is any 2×2 unitary matrix that changes the qubit state and which can be visualized as a transition from one point on the Bloch sphere to another. A unitary matrix preserves the norm and maps a unit vector into another unit vector – this is why the 1-qubit gates are often referred to as rotations.

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Figure A1: Schematic representation of the Bloch sphere with rotation around the *z*-axis



In the computational basis, the North pole of the Bloch sphere ($\theta = 0$ in expression 3) corresponds to state $|0\rangle$:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and the South pole of the Bloch sphere ($\theta = \pi$ in expression 3) corresponds to state $|1\rangle$:

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Therefore, an arbitrary qubit state $|\psi\rangle$ can be represented by a superposition of the basis states $|0\rangle$ and $|1\rangle$:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

$$\alpha = \cos\left(\frac{\theta}{2}\right), \quad \beta = e^{i\phi}\sin\left(\frac{\theta}{2}\right),$$

where α and β are probability amplitudes. The probability of finding a qubit in state $|0\rangle$ ($|1\rangle$) after measurement in the computational basis is given by $|\alpha|^2$ ($|\beta|^2$).

There are infinitely many possible 1-qubit gates (rotations), but some of them are more useful than others. For example, we can easily verify by direct calculation that Pauli X – rotation by π radians around the x-axis – implements the quantum NOT gate that flips the qubit state:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and Pauli Z – rotation by π radians around the z-axis – flips the phase:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = -\begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A very useful class of 2-qubit gates is *controlled rotation*. A controlled rotation is a rotation applied to the target qubit if the control qubit is in state $|1\rangle$. The state of a 2-qubit system is represented by the four-dimensional unit vector and all 2-qubit gates can be represented by the 4×4 unitary matrices. For example, the controlled NOT (CX) gate has the form

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

and controlled rotation around the z-axis by angle ϕ (controlled phase shift gate) is given by the following unitary matrix:

$$CR_z(\phi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\phi} \end{bmatrix}.$$

15418286, 2021, 114, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/wilm.10943 by CBS Library, Wiley Online Library on [31/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License